Format des données

Table des matières

1 Généralités 1
2 Les commandes 1
3 Les entiers posififs ou nuls 1
4 Les entiers négatifs 2
5 Les chaînes de caractères Unicode 3
5.1 Codage des caractères ASCII 3
5.2 Codage des points de code jusqu'à 10 bits 3
5.3 Codage des points de code jusqu'à 16 bits 4
5.4 Codage des points de code jusqu'à 22 bits 4
5.5 Encodage d'une chaîne de caractères 4
6 Flottants 32 bits 6

1 Généralités

Le format est conçu de façon à pouvoir transmettre des séquences de *commandes*, à chacune d'elle sont associés zéro, un ou plusieurs *paramètres* qui peuvent être des entiers non signés, des entiers signés et des chaînes de caractères Unicode.

C'est un format *binaire* de type *autosynchronisant*, qui contient des séquences de *commandes* précédées par des *paramètres*. Les paramètres sont placés avant la commande à laquelle ils sont associés (notation *polonaise inverse*).

Le format est une chaîne d'octets. Chaque octet transmis obéit à ce format :

- $0xxx$ _ $xxxx$: extension de valeur ;
- 1000 _ $xxxx$: entier positif ou nul ;
- 1001 _ $xxxx$: entier négatif ;
- 1010 _ 0000 : chaîne de caractères Unicode ;
- 1010 _ $xxxx$, avec $x \neq 0$: réservés à des extensions futures ;
- 1011 _ $xxxx$, flottant 32 bits, au format IEEE 754 ;
- $11xx$ _ $xxxx$: commande.

2 Les commandes

Il y a 64 commandes possibles codées $11c_5c_4c_3c_2c_1c_0$. La signification d'une commande est différente selon le sens.

3 Les entiers posififs ou nuls

Un entier non signé est défini par un octet 1000 _ $b_3b_2b_1b_0$ qui contient les quatre bits de poids faible de la valeur à coder. Si cette valeur est supérieure à 15, une ou plusieurs *exten-*

sions de valeur sont ajoutées. Chacune d'elles contient sept bits qui, décalés, contiennent les bits supplémentaires. Ce format favorise le codage compacte des faibles valeurs.

Une valeur sur 11 bits est codée sur deux octets :

$$1000\ b_3b_2b_1b_0 \quad 0b_{10}b_9b_8b_7\ b_6b_5b_4$$

Une valeur sur 18 bits est codée sur trois octets :

$$1000\ b_3b_2b_1b_0 \quad 0b_{10}b_9b_8\ b_7b_6b_5b_4 \quad 0b_{17}b_{16}b_{15}\ b_{14}b_{13}b_{12}b_{11}$$

Une valeur sur 25 bits est codée sur quatre octets :

$$1000\ b_3b_2b_1b_0 \quad 0b_{10}b_9b_8\ b_7b_6b_5b_4$$

$$0b_{17}b_{16}b_{15}\ b_{14}b_{13}b_{12}b_{11} \quad 0b_{24}b_{23}b_{22}\ b_{21}b_{20}b_{19}b_{18}$$

Une valeur sur 32 bits est codée sur cinq octets :

$$1000\ b_3b_2b_1b_0 \quad 0b_{10}b_9b_8\ b_7b_6b_5b_4$$

$$0b_{17}b_{16}b_{15}\ b_{14}b_{13}b_{12}b_{11} \quad 0b_{24}b_{23}b_{22}\ b_{21}b_{20}b_{19}b_{18}$$

$$0b_{31}b_{30}b_{29}\ b_{28}b_{27}b_{26}b_{25}$$

L'encodage des valeurs entières peut être réalisé par la fonction C++ suivante :

```
void encodeU32 (std::vector <uint8_t> & ioBuffer,
                const uint32_t inValue) {
  ioBuffer.push_back (uint8_t ((inValue & 0x0F) | 0x80)) ;
  uint32_t v = inValue >> 4 ;
  while (v > 0) {
    ioBuffer.push_back (uint8_t (v & 0x7F)) ;
    v >>= 7 ;
  }
}
```

Voici quelques exemples d'encodage :

Décimal	Hex	Code
0	0	80
100	64	84 06
1023	3FF	8F 3F
60000000	393_8700	80 70 70 64 01
536870911	1FFF_FFFF	8F 7F 7F 7F 0F
4294967295	FFFF_FFFF	8F 7F 7F 7F 7F
18446744073709551615	FFFF_FFFF_FFFF_FFFF	8F 7F 7F 7F 7F 7F 7F 7F 7F 0F

4 Les entiers négatifs

C'est leur complément à 1 qui est codé comme un entier positif ou nul, l'octet initial étant $1001\ _\ xxxx$.

Ansi, pour la valeur –1 : le complément à 1 est 0, son codage est $1001\ 0000$.

L'encodage des valeurs signées peut être réalisé par la fonction C++ suivante :

```
void encodeS32 (std::vector <uint8_t> & ioBuffer,
                const int32_t inValue) {
  if (inValue >= 0) {
    encodeU32 (ioBuffer, uint32_t (inValue)) ;
  }else{
    uint32_t v = ~ uint32_t (inValue) ;
    ioBuffer.push_back (uint8_t ((v & 0x0F) | 0x90)) ;
    v >>= 4 ;
    while (v > 0) {
      ioBuffer.push_back (uint8_t (v & 0x7F)) ;
      v >>= 7 ;
    }
  }
}
```

5 Les chaînes de caractères Unicode

Le codage d'une chaîne de caractères commence par l'octet $1010\ _\ 0000$. Tel que, ceci code la chaîne vide. Les octets *extensions de valeur* qui suivent fournissent les caractères de la chaîne.

Un point de code Unicode est une valeur sur 21 bits que l'on note

$$u_{20}\ u_{19}u_{18}u_{17}u_{16}\ \ u_{15}u_{14}u_{13}u_{12}\ u_{11}u_{10}u_9u_8\ \ u_7u_6u_5u_4\ u_3u_2u_1u_0$$

5.1 Codage des caractères ASCII

Un caractère ASCII est codé directement par $0u_6u_5u_4\ u_3u_2u_1u_0$, avec une exception : les valeurs entre $0x10$ et $0x1F$ forment le premier octet du codage des caractères Unicode autres que ASCII et de ceux compris entre $0x10$ et $0x1F$.

5.2 Codage des points de code jusqu'à 10 bits

$$0001\ u_3u_2u_1u_0$$

$$01u_9u_8\ u_7u_6u_5u_4$$

Ce codage est aussi utilisé pour les caractères entre $0x10$ et $0x1F$.

- $0x10$ est codé $0001\ 0000\ \ 0100\ 0001$;
- $0x1F$ est codé $0001\ 1111\ \ 0100\ 0001$.

Il permet aussi de représenter tous les caractères latins accentués :

- « à », point de code 0xE0, codé par $0001\ 0000\ \ 0100\ 1110$;
- « œ », point de code 0x153, codé par $0001\ 0011\ \ 0101\ 0101$;

5.3 Codage des points de code jusqu'à 16 bits

$$0001\ u_3u_2u_1u_0$$

$$00u_9u_8\ u_7u_6u_5u_4$$

$$01u_{15}u_{14}u_{13}u_{12}\ u_{11}u_{10}$$

5.4 Codage des points de code jusqu'à 22 bits

Soit un bit de plus que nécessaire.

$$0001\ u_3u_2u_1u_0$$

$$00u_9u_8\ u_7u_6u_5u_4$$

$$00u_{15}u_{14}\ u_{13}u_{12}u_{11}u_{10}$$

$$01u_{21}u_{20}\ u_{19}u_{18}u_{17}u_{16}$$

Ensuite, on encode la chaîne de caractères :

5.5 Encodage d'une chaîne de caractères

Comme la chaîne est encodée en UTF-8, il faut d'abord extraire les points de code. La fonction suivante fait cette extraction :

```
uint32_t utf32CharacterForPointer (const uint8_t * inDataString,
                                   size_t & ioIndex,
                                   const size_t inLength,
                                   bool & ioOK) {
  uint32_t result = 0 ;
  uint32_t c = inDataString [ioIndex] ;
  ioIndex += 1 ;
  ioOK = true ;
  if ((c & 0x80) == 0) {
    result = c ;
  }else if ((c & 0xE0) == 0xC0) {
    result = c & 0x1F ;
    result <<= 6 ;
    c = inDataString [ioIndex] ;
    ioOK = ((c & 0xC0) == 0x80) && (ioIndex < inLength) ;
    if (ioOK) {
      ioIndex += 1 ;
      result |= c & 0x3F ;
    }
  }else if ((c & 0xF0) == 0xE0) {
    result = c & 0x0F ;
    result <<= 12 ;
    c = inDataString [ioIndex] ;
    ioOK = ((c & 0xC0) == 0x80) && (ioIndex < inLength) ;
    if (ioOK) {
      ioIndex += 1 ;
      result |= (c & 0x3F) << 6 ;
      c = inDataString [ioIndex] ;
      if (ioOK) {
```

```
      ioOK &= ((c & 0xC0) == 0x80) && (ioIndex < inLength) ;
      ioIndex += 1 ;
      result |= c & 0x3F ;
    }
  }
 }else if ((c & 0xF8) == 0xF0) {
   result = (c & 0x07) << 18 ;
   c = inDataString [ioIndex] ;
   ioOK = ((c & 0xC0) == 0x80) && (ioIndex < inLength) ;
   if (ioOK) {
     ioIndex += 1 ;
     result |= (c & 0x3F) << 12 ;
     c = inDataString [ioIndex] ;
     ioOK = ((c & 0xC0) == 0x80) && (ioIndex < inLength) ;
     if (ioOK) {
       ioIndex += 1 ;
       result |= (c & 0x3F) << 6 ;
       c = inDataString [ioIndex] ;
       ioOK = ((c & 0xC0) == 0x80) && (ioIndex < inLength) ;
       if (ioOK) {
         ioIndex += 1 ;
         result |= c & 0x3F ;
       }
     }
   }
 }else{
   ioOK = false ;
 }
 if (!ioOK) {
   result = 0x0000FFFD ; // UNICODE_REPLACEMENT_CHARACTER
 }
 return result ;
}
```

Rappel du codage UTF-8 :

Point de code	**UTF-8**
$0\ 0000\ \ 0000\ 0000\ \ 0u_6u_5u_4\ u_3u_2u_1u_0$	$0u_6u_5u_4\ \ u_3u_2u_1u_0$
$0\ 0000\ \ 0000\ 0u_{10}u_9u_8\ \ u_7u_6u_5u_4\ u_3u_2u_1u_0$	$110u_{10}\ u_9u_8u_7u_6$
	$10u_5u_4\ \ u_3u_2u_1u_0$
$0\ 0000\ \ u_{15}u_{14}u_{13}u_{12}\ u_{11}u_{10}u_9u_8\ \ u_7u_6u_5u_4\ u_3u_2u_1u_0$	$1110\ u_{15}u_{14}u_{13}u_{12}$
	$10u_{11}u_{10}\ \ u_9u_8u_7u_6$
	$10u_5u_4\ \ u_3u_2u_1u_0$
$u_{20}\ u_{19}u_{18}u_{17}u_{16}\ \ u_{15}u_{14}u_{13}u_{12}\ u_{11}u_{10}u_9u_8\ \ u_7u_6u_5u_4\ u_3u_2u_1u_0$	$1111\ \ 0u_{20}u_{19}u_{18}$
	$10u_{17}u_{16}\ \ u_{15}u_{14}u_{13}u_{12}$
	$10u_{11}u_{10}\ \ u_9u_8u_7u_6$
	$10u_5u_4\ \ u_3u_2u_1u_0$

Le codage d'une chaîne UTF-8 est réalisé par la fonction :

```
void encodeStr (std::vector <uint8_t> & ioBuffer,
                const std::string inUTF8String) {
  ioBuffer.push_back (0xA0) ;
  const uint8_t * cString = (const uint8_t *) inUTF8String.c_str () ;
  const size_t utf8ByteCount = strlen ((const char *) cString) ;
  size_t utf8Index = 0 ;
  bool ok = true ;
```

```
  while ((utf8Index < utf8ByteCount) && ok) {
    const uint32_t unicodeChar = utf32CharacterForPointer (cString, utf8Index, utf8ByteCount,
ok) ;
    std::cout << "    encoded utf32 : 0x" << std::hex << unicodeChar << std::dec << "\n" ;
    if (ok) {
      if ((unicodeChar >= 0x10) && (unicodeChar <= 0x1F)) { // control 0001 xxxx
        ioBuffer.push_back (uint8_t (unicodeChar)) ;
        ioBuffer.push_back (0x41) ;
      }else if (unicodeChar <= 0x7F) { // ASCII
        ioBuffer.push_back (uint8_t (unicodeChar)) ;
      }else if (unicodeChar <= 0x3FF) {
        ioBuffer.push_back (uint8_t (unicodeChar & 0x0F) | 0x10) ;
        ioBuffer.push_back (uint8_t (unicodeChar >> 4) | 0x40) ;
      }else if (unicodeChar <= 0xFFFF) {
        ioBuffer.push_back (uint8_t (unicodeChar & 0x0F) | 0x10) ;
        ioBuffer.push_back (uint8_t (unicodeChar >> 4) & 0x3F) ;
        ioBuffer.push_back (uint8_t (unicodeChar >> 10) | 0x40) ;
      }else{
        ioBuffer.push_back (uint8_t (unicodeChar & 0x0F) | 0x10) ;
        ioBuffer.push_back (uint8_t (unicodeChar >> 4) & 0x3F) ;
        ioBuffer.push_back (uint8_t (unicodeChar >> 10) & 0x3F) ;
        ioBuffer.push_back (uint8_t (unicodeChar >> 16) | 0x40) ;
      }
    }
  }
}
```

6 Flottants 32 bits

Un flottant 32 bits au format IEEE 754[1] est composé d'un bit de signe, 8 bits pour l'exposant et 23 pour la mantisse :

$$s\ e_7 ... e_0\ m_{22} ... m_0$$

Un flottant 32 bits est encodé sur 5 octets à partir des ports forts :

$$\mathbf{1\ 0\ 1\ 1}\ \ s\ e_7\ e_6\ e_5$$

$$\mathbf{0}\ e_4\ e_3\ e_2\ \ e_1\ e_0\ m_{22}\ m_{21}$$

$$\mathbf{0}\ m_{20}\ m_{19}\ m_{18}\ \ m_{17}\ m_{16}\ m_{15}\ m_{14}$$

$$\mathbf{0}\ m_{13}\ m_{12}\ m_{11}\ m_{10}\ \ m_9\ m_8\ m_7$$

$$\mathbf{0}\ m_6\ m_5\ m_4\ \ m_3\ m_2\ m_1\ m_0$$

Les octets nuls en fin sont éliminés. Par exemple $0,15625$ est représenté en IEEE 754 par $0\ 01111100\ 01000000000000000000000$, cette valeur est codée sur un seul octet :

$$\mathbf{1\ 0\ 1\ 1}\ \ 0\ 1\ 0\ 0$$

Il en est de même pour +0 qui est codé par $\mathbf{1\ 0\ 1\ 1}\ \ 0\ 0\ 0\ 0$ et –0 qui est codé par $\mathbf{1\ 0\ 1\ 1}\ \ 1\ 0\ 0\ 0$.

L'encodage des valeurs flottantes 32 bits peut être réalisé par la fonction C++ suivante :

[1] https://fr.wikipedia.org/wiki/IEEE_754

```
void encodeF32 (std::vector <uint8_t> & ioBuffer,
                const float inValue) {
  ioBuffer.push_back (uint8_t ((inValue & 0x0F) | 0x80)) ;
  uint32_t v = inValue >> 4 ;
  while (v > 0) {
    ioBuffer.push_back (uint8_t (v & 0x7F)) ;
    v >>= 7 ;
  }
}
```